

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO5: Construct interoperable web services by leveraging JAX-RPC, WSDL, SOAP, and XML Schema for seamless data exchange across distributed systems.(L4,L5)

Assessment Tool : Alternative Solution Design

Weightage: 20%

QUESTIONS:

Total Marks: 25

- A company needs to develop a web service for a mobile and web application. Analyze both **REST and SOAP** approaches and design an appropriate solution by selecting one. Justify your choice based on performance, scalability, and security requirements.
- An online platform expects a large number of users accessing its services simultaneously. Design a **scalable service architecture** showing how client, server, API, and database interact. Propose an alternative design to improve scalability and justify your approach.
- A banking application requires secure communication between client and server. Analyze available communication protocols and **select the most suitable protocol**. Justify your selection based on security, reliability, and performance.
- An application needs to integrate with third-party services such as payment gateways or maps. Design an **API integration strategy**, including request handling, authentication, and response processing. Suggest an alternative approach to improve efficiency or reliability.
- A web service frequently encounters errors during client requests. Design an **error handling and fault tolerance mechanism**. Propose alternative strategies to improve system reliability and ensure smooth user experience.

Code of Conduct:

I Aaron Samuel S(192411022) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weight age (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly defines problem, objectives, and all requirements accurately	Understands problem with minor gaps	Partial understanding; misses key requirements	Misinterprets or fails to understand problem
Generation of Solutions	25%	Develops multiple innovative and feasible alternatives with clear differentiation	Generates relevant alternatives with minor limitations	Limited or repetitive alternatives	Fails to generate meaningful alternatives
Evaluation of Alternatives	25%	Critically compares alternatives using appropriate criteria and metrics	Compares alternatives with some justification	Basic or superficial comparison	No proper comparison conducted
Solution Selection	20%	Selects best solution with strong justification and decision rationale	Selects suitable solution with moderate justification	Selection is weakly justified	No clear or justified selection

1. REST vs SOAP for a Mobile and Web Application

Analysis

REST (Representational State Transfer) is an architectural style that commonly uses HTTP methods such as GET, POST, PUT, and DELETE. It usually exchanges lightweight JSON data, making it well suited for mobile and web applications.

SOAP (Simple Object Access Protocol) is a protocol that uses XML-based messages. It provides built-in standards for features such as formal contracts, enterprise security, transactions, and reliable messaging.

Factor REST SOAP

Performance Fast; lightweight JSON Generally slower because of XML overhead

Scalability Highly suitable for distributed/cloud systems More complex to scale

Mobile support Excellent Less suitable because of message size

Security HTTPS, OAuth 2.0, JWT, API gateways WS-Security and HTTPS

Flexibility High More rigid and contract-oriented

Complexity Relatively simple More complex

Best use Web/mobile APIs Enterprise transactions and legacy systems

Selected solution: REST

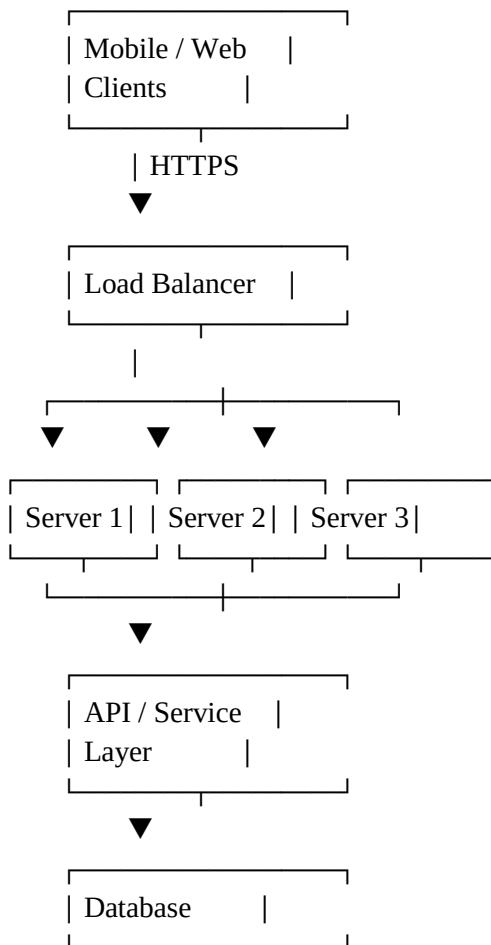
I would select REST over HTTPS for the application. Mobile and web clients can communicate with REST endpoints using JSON, reducing bandwidth and processing requirements. REST's stateless nature also makes it easier to scale horizontally by adding additional application servers.

Security can be implemented using TLS/HTTPS, OAuth 2.0 or OpenID Connect for authentication, short-lived access tokens, authorization controls, input validation, rate limiting, and an API gateway.

Conclusion: REST provides better performance, scalability, and development flexibility for a modern mobile and web application. SOAP would be preferable if the system had strict enterprise requirements such as WS-Security, transactional standards, or integration with existing SOAP-based systems.

2. Scalable Architecture for a Large Number of Simultaneous Users

Basic architecture

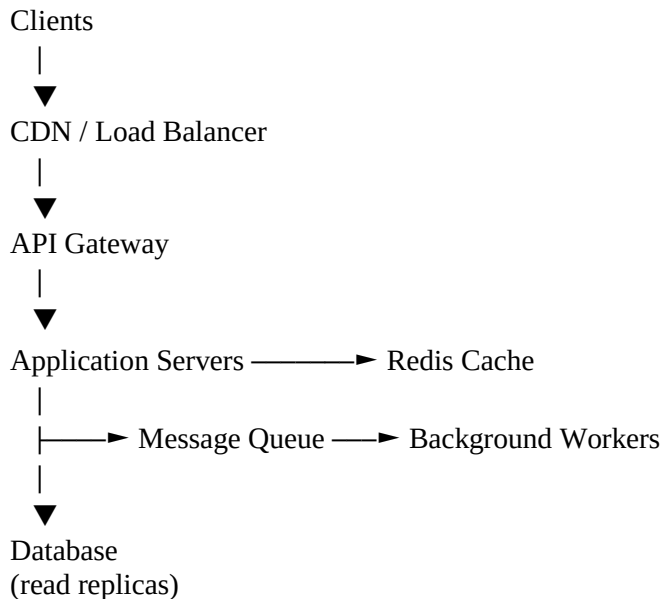


Clients send HTTPS requests to a load balancer, which distributes traffic among multiple stateless application servers. The API/service layer processes requests and communicates with the database.

The architecture should use connection pooling, database indexes, pagination, caching, monitoring, and automatic health checks. Servers can be added or removed according to demand.

Alternative design: cache + asynchronous processing

A more scalable architecture can introduce a distributed cache and message queue:



A cache such as Redis can serve frequently requested data without repeatedly accessing the database. A message queue can move slow operations—such as email processing, report generation, or notifications—to background workers.

Justification: This design reduces database load, prevents slow operations from blocking users, and allows each component to scale independently. Database read replicas can further increase read capacity.

3. Secure Communication for a Banking Application

Several protocols can be considered:

HTTP: Simple but unsuitable for transmitting sensitive banking information because it does not encrypt traffic.

HTTPS: HTTP protected by TLS. Provides encryption, server authentication, and integrity protection.

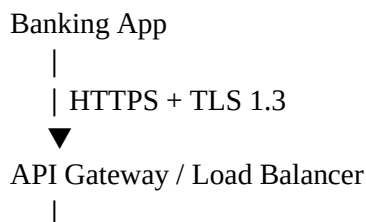
TLS: The underlying security protocol used to protect application communication.

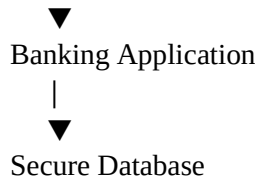
SSH: Excellent for secure administration and server-to-server tasks, but not normally used for browser/mobile application APIs.

WebSocket over TLS (WSS): Useful when a persistent, real-time two-way connection is required, but unnecessary for ordinary request-response banking APIs.

Selected protocol: HTTPS with TLS 1.3

For normal banking API communication, I would use HTTPS with TLS 1.3.





Security: TLS encrypts data in transit and protects against eavesdropping and tampering. Certificates authenticate the server. Strong authentication such as OAuth 2.0/OpenID Connect, MFA, and short-lived tokens should be used at the application layer.

Reliability: HTTPS runs over TCP, providing reliable, ordered delivery. The application should additionally implement transaction controls, timeouts, retries where safe, and idempotency for operations such as payments.

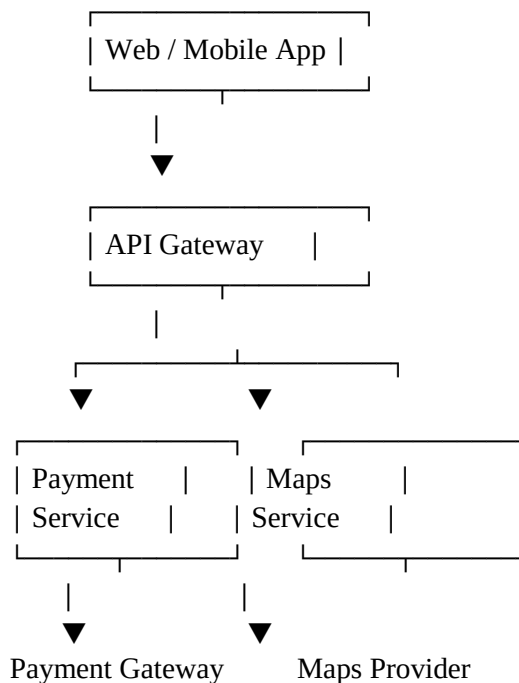
Performance: TLS 1.3 reduces handshake overhead compared with older TLS versions, while HTTP/2 or HTTP/3 can improve connection efficiency.

Conclusion: HTTPS with modern TLS is the appropriate standard for banking client-server communication because it combines strong confidentiality, integrity, authentication, reliability, and good performance.

4. API Integration with Third-Party Services

Suppose an application integrates with a payment gateway and a maps service.

Integration architecture



Request handling

The client sends a request to the application's API.

The API authenticates and authorizes the user.

The appropriate internal service validates the request.

The service creates a request for the third-party API.

Authentication credentials are securely attached.

The third-party response is validated and converted into the application's standard response format.

The result is returned to the client.

Authentication and security

API keys, OAuth 2.0 access tokens, or signed requests should be used according to the third-party provider's requirements. Secrets should be stored in a secure secret-management system rather than source code.

Sensitive information should be transmitted only through HTTPS. Input validation, rate limiting, audit logging, and access controls should also be applied.

Error and response processing

The application should distinguish between:

Successful responses.

Authentication/authorization failures.

Invalid requests.

Rate-limit errors.

Temporary provider failures.

Timeouts.

Permanent business errors.

Alternative approach: asynchronous integration

For operations that do not need an immediate result, use a message queue:

Application



Message Queue



Integration Worker



Third-Party API



Result / Event

This prevents temporary third-party failures from directly affecting users. Workers can retry failed requests using exponential backoff and a dead-letter queue.

Justification: Synchronous API calls are appropriate when the user needs an immediate response, while asynchronous processing improves reliability and scalability for long-running or non-critical operations.

5. Error Handling and Fault Tolerance

A web service should handle failures at multiple levels rather than allowing errors to propagate directly to users.

Proposed mechanism

Client



API Gateway



Application Servers



— Timeout

— Retry

— Circuit Breaker

— Validation

— Error Logging



Database / External Services

Error-handling strategy

Input validation: Reject invalid requests before expensive processing.

Standard error responses: Return consistent HTTP status codes such as 400, 401, 403, 404, 409, 429, and 500.

Timeouts: Prevent a failed dependency from keeping resources occupied indefinitely.

Retries: Retry temporary failures using exponential backoff and jitter.

Idempotency: Ensure operations such as payment requests are not accidentally executed twice during retries.

Circuit breaker: Temporarily stop calling an unhealthy dependency to prevent cascading failures.

Fallback: Return cached or reduced functionality where appropriate.

Logging and monitoring: Record errors with correlation IDs so individual requests can be traced.

Health checks: Remove unhealthy servers from load-balancer rotation.

Graceful degradation: Provide a useful response even when a non-essential service is unavailable.

Alternative strategies

Active-active deployment: Run application instances across multiple availability zones or regions. If one location fails, traffic can be redirected to another.

Queue-based architecture: Put non-critical operations into a message queue so temporary downstream failures do not immediately affect the user.

Caching: Serve recently stored data when the primary service is temporarily unavailable.

Bulkheads: Isolate resources for different services so failure in one component does not consume all system resources.